Hart, Noah - u1537288 - Actively Contributing

(Level) Design

This sprint, 75% of the time was spent on Design. I created a master level design doc and finalized the design layout for the train station, and met with Yukon on how the general flow would be. I met with Cleveland to discuss the narrative elements and why the player wakes up with enemies in the starting train car, but they don't respawn when the player dies, while all the other enemies do. I also finalized the design for the “safe house,” which serves as the lobby and spawn point for the player to upgrade their meta progression and equip their desired weapons before they start their run. The train will change as you get further, and also explore different, more challenging encounters for the last few train phases. Lastly, I met with Richard and Cleveland to go over what the player will keep and lose on each run and overall metaprogression. Things like total amount of enemies killed, encounters completed, and farthest distance achieved all factor into what the player can upgrade, with the focus being on minor unlocks or upgrades like craft recipes and small damage/health bonuses, to have the player progress farther with each run they do.

MetaProgression - Things kept/lost when player dies

Keep	Lose
XP	Crafting items
Reciepes	Weapon mods and ammo
Rewards from Puzzles	Crafted items
Prema Buffs	Buffs

Puzzles will be one-time completes as we want to balance player progression with the thought that they will be back at these train stations, so that next run throug,h they can focus on one facet of either exploring or unlocking new gear (typically first time visits), also it is not possible to procedurally generate quality or challenging puzzles and players will already know the solution. To give risk, reward, and pressure to the player to get back on the train, we will be setting some time limits which help the player make strategic and meaningful choices, and since the gameplay is highly replayable, we don’t want to give too much time at stations, otherwise the player will want to skip them because they have it figured out already.

Engineering

Engineering side, I met with those working on the engine to discuss naming conventions and file structure. Programming-wise, I continued putting in the framework to achieve a working gameplay loop. The carts now spawn randomly with a couple of rules:

- Train Phase - Where the player navigates through 3-4 randomly spawned train cars, which serves as the main rogue-like gameplay loop
    - Cars are picked from an array of preset layouts with 3 templates as a basis for now
        - The factors of the templates are primarily length and height
        - Cars have snap points so that the next car knows where to spawn
    - Encounters, Challenges, and modifiers
        - In the manager script, there is a list of encounters that can spawn with challenges and modifiers with various degrees of weighting
            - There is also an idea of making specific rooms tailored to specific challenges, so either treat them as special rooms or just tailor the pool of encounters/challenges/modifiers for those rooms specifically
- TrainStation Phase - Player explores, solves puzzles, and can craft with items from the loot they scavenge
    - Limited time to loot before the train announces it will depart
    - The player essentially loses if they miss the train, but we will have enemies continuously spawn until the player dies. This could give a small amount of XP for meta progression upgrades

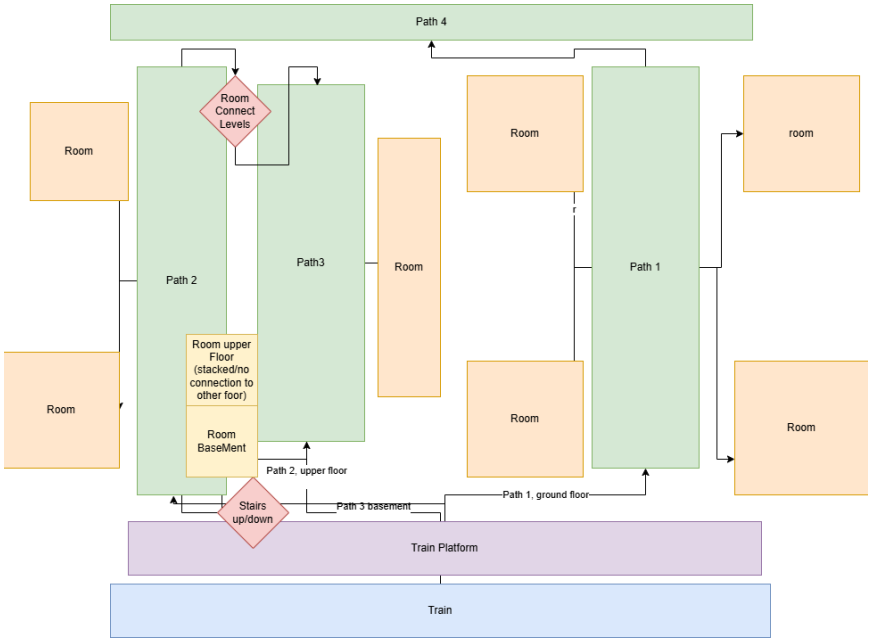
The framework is in place; I just need to hook up and test in the editor for the most part. The idea is that we have the 4 main types of levels. The tutorial level, the train station (technically more than one, depending on the total number of stations we have the player visit, and the train cars (streamed levels that load multiple pre-made levels, final boss is part of this, but will be more of a special case for the final train phase), and the player safe hub/spawn

Looking Forward

- Full gameplay loop done by mid-sprint
    - The player can go through a full train phase and loot a train station. Dying sends them to the spawn car
    - Next door opens depending on if it's the last car in sequence to send to the next encounter or a train station

- Tutorial level - first car and train station
- More train car layouts are designed

Rooms Generated in different orders ^




```cpp
void AEncounterManager::OnCarStreamed(ULevelStreamingDynamic* StreamedLevel,
    const TSoftObjectPtr<UWorld> CarLevel,
    int32 CarIdx)
{
    if (!StreamedLevel)
    {
        UE_LOG(LogTemp, Error, TEXT("OnCarStreamed: StreamedLevel null for car %d"), CarIdx);
        return;
    }

    FVector NextAttachLocation = CurrentBuildLocation;
    bool bFoundSnap = false;

    if (SnapPointClass)
    {
        TArray<AActor*> SnapPoints;
        UGameplayStatics::GetAllActorsOfClass(GetWorld(), SnapPointClass, SnapPoints);

        UE_LOG(LogTemp, Log, TEXT("Searching %d snap points for car %d (%s)"),
            SnapPoints.Num(),
            CarIdx + 1,
            *CarLevel.GetAssetName());

        for (AActor* Point : SnapPoints)
        {
            if (!Point) continue;

            if (Point->GetLevel() == StreamedLevel->GetLoadedLevel())
            {
                bFoundSnap = true;
                NextAttachLocation = Point->GetActorLocation();
                UE_LOG(LogTemp, Warning, TEXT("SnapPoint matched for %s at %s"),
                    *CarLevel.GetAssetName(),
                    *NextAttachLocation.ToString());
                break;
            }
        }
    }
```

```cpp
void AEncounterManager::BeginBuildTrain()
{
    // End condition
    if (BuildCarIndex >= SelectedTrainLevels.Num())
    {
        UE_LOG(LogTemp, Log, TEXT("All %d train cars built successfully."), SelectedTrainLevels.Num());
        return;
    }

    UE_LOG(LogTemp, Log, TEXT("Spawning car %d of %d at %s"),
        BuildCarIndex + 1,
        SelectedTrainLevels.Num(),
        *CurrentBuildLocation.ToString());

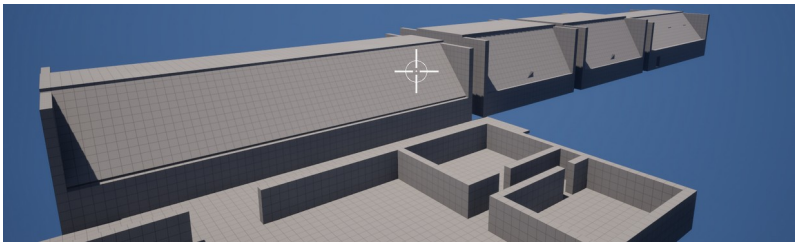
    StreamCarAtIndex(BuildCarIndex);
}

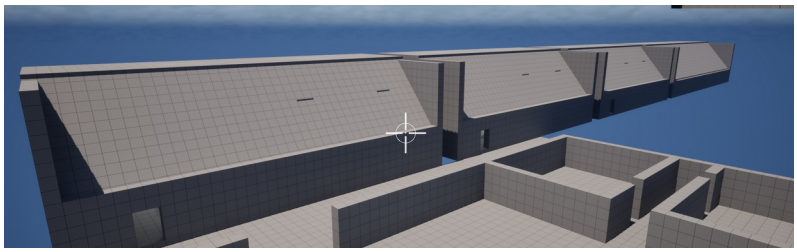
void AEncounterManager::StreamCarAtIndex(int32 CarIdx)
{
    if (!SelectedTrainLevels.IsValidIndex(CarIdx))
    {
        UE_LOG(LogTemp, Error, TEXT("StreamCarAtIndex: invalid index %d"), CarIdx);
        return;
    }

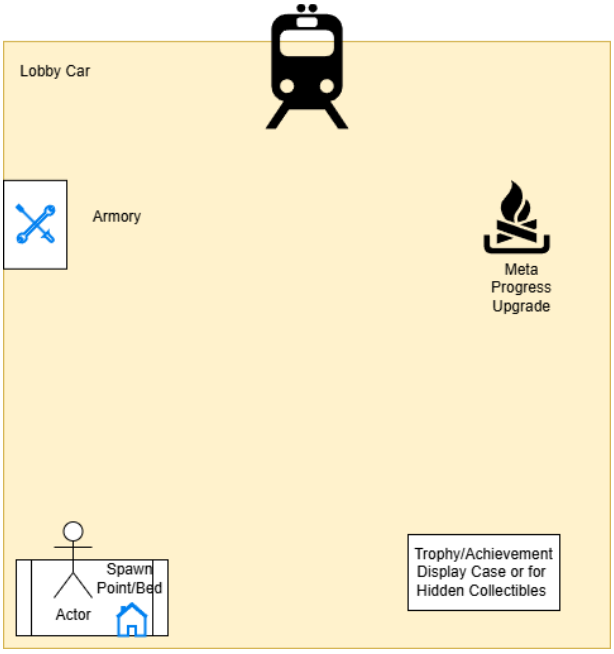
    const TSoftObjectPtr<UWorld>& CarLevel = SelectedTrainLevels[CarIdx];
    bool bSuccess = false;

    // Give every instance a unique name so UE can load the same map multiple times
    const FString UniqueInstanceName = FString::Printf(TEXT("%s_Instance_%d_%d"),
        *CarLevel.GetAssetName(),
        CarIdx,
        FMath::RandRange(1000, 9999));

    ULevelStreamingDynamic* StreamedLevel = ULevelStreamingDynamic::LoadLevelInstanceBySoftObjectPtr(
        GetWorld(),
        CarLevel,
        CurrentBuildLocation,
        FRotator::ZeroRotator,
        bSuccess
    );
```






Lobby Car
Armory
Meta
Progress
Upgrade
Spawn
Point/Bed
Actor
Trophy/Achievement
Display Case or for
Hidden Collectibles